Supplementary Material: Computer-Assisted Accuracy Proof

Version of 2026-10-01

1 Computer-assisted accuracy proof

1.1 How to read this supplement

This supplement is written as a self-contained computer-assisted-proof appendix. Each numerical claim is tied to a mathematical statement, the Julia routine that produced the enclosure, the Lean module that checks the corresponding logical consequence, and the archived Crunchy2 artifact from which the final constants were taken. The organization is intentionally more explicit than a short Methods section: the goal is to make the boundary between discovery, interval computation, formal proof, and remaining assumptions auditable by a reader.

The main theorem exported by the supplement is a residual certificate for a single finite cosine series. The theorem is not a nearby-solution theorem, and no step in this document asserts an inverse estimate for the linearized Babenko operator. The supplement therefore records exactly what has been certified and exactly what would be needed for a later existence upgrade.

1.2 Statement and scope of the certificate

The purpose of the computer-assisted calculation is to certify the accuracy of one explicit numerical traveling-wave profile. After the search and polishing stages, the profile is frozen as the finite cosine polynomial

$$y_N(u) = \frac{a_0}{2} + \sum_{k=1}^{N} a_k \cos(ku), \qquad N = 150. \tag{1}$$

The coefficients in Eq. (1) are fixed binary floating-point numbers. The verification treats their exact dyadic values as the definition of the candidate; it does not assume that they approximate unknown ideal coefficients. The neural network is therefore a search tool, not part of the object appearing in the theorem.

Let H denote the 2π-periodic Hilbert transform with convention

$$H[\cos(ku)] = \sin(ku), \qquad H[\sin(ku)] = -\cos(ku), \qquad H[1] = 0. \tag{2}$$

For

$$D = \sqrt{y_u^2 + \big(1 + H[y_u]\big)^2}, \qquad G = \frac{1 + H[y_u]}{D}, \tag{3}$$

the residual evaluated by the verifier is

$$\begin{aligned} R(y) = {} & \mu H[y_u] - y - yH[y_u] - H[yy_u] \\ & + \beta\, \partial_u\Big(\frac{y_u}{D}\Big) - \beta H[\partial_u G]. \end{aligned} \tag{4}$$

The PINN uses the parameters c and b, whereas Eq. (4) uses

$$\mu = \mathrm{fl}_{64}(c^2) = 1.3660618372350841, \tag{5}$$

$$\beta = \mathrm{fl}_{64}(b^2) = 0.248004, \tag{6}$$

$$c = 1.1687864806007486, \qquad b = 0.498. \tag{7}$$

Here fl_{64} denotes the stored IEEE double-precision result of the indicated operation. The certificate later records those stored values as exact dyadic rationals. This squaring is part of the certificate specification. Substituting the unsquared PINN parameters into Eq. (4) would verify a different equation.

The calculation proves two continuum inequalities:

$$\inf_{u\in\mathbb{R}} D(u) > 0, \qquad \sup_{u\in\mathbb{R}} |R(y_N)(u)| \le \varepsilon_*. \tag{8}$$

For the successful run,

$$D(u) \ge 0.3446588, \qquad |R(y_{150})(u)| \le 1.78 \times 10^{-13} \quad \text{for every } u \in \mathbb{R}. \tag{9}$$

The manuscript Babenko operator $\mathcal{B}$ uses the opposite residual sign in the normalization $g = 1$:

$$R(y_N)(u) = -\mathcal{B}(y_N)(u). \tag{10}$$

Consequently Eq. (9) is also the claimed bound for $|\mathcal{B}(y_{150})|$.

This is an accuracy theorem for a supplied finite series. It is not an existence or uniqueness theorem for an exact nearby water wave. Such a result would additionally require a rigorous inverse bound for the linearized Babenko operator and a Newton–Kantorovich or contraction argument; neither ingredient is part of the present certificate.

1.3 Certified statement and proof entry points

Supplementary Theorem 1 (Crunchy2 phase–4 $N = 150$ residual certificate)**.** *Let y_{150} be the finite cosine polynomial in Eq.* (1)*, with the 151 coefficients stored in* `CertificatePhase4ScalarN150Phase420260827HalfDC.lean`*. Let $\mu = 1.3660618372350841$ and $\beta = 0.248004$ be the exact dyadic values recorded in the same certificate. Then the denominator D from Eq.* (3) *is positive for every $u \in \mathbb{R}$, and*

$$\forall u \in \mathbb{R}, \qquad |R(y_{150})(u)| \le \frac{178}{10^{15}}. \tag{11}$$

Equivalently, for the manuscript Babenko operator $\mathcal{B}$ in the normalization $g = 1$,

$$\forall u \in \mathbb{R}, \qquad |\mathcal{B}(y_{150})(u)| \le \frac{178}{10^{15}}. \tag{12}$$

Proof architecture. The profile and parameters are fixed as exact rational data in the generated Lean certificate. Julia/Arb proves the denominator and residual anchors A_D and A_R described in Eqs. (60)–(61). Lean then checks the denominator corollary, the residual theorem on one period, the periodic extension to all real u, the exact rational comparison $\varepsilon_{0,\mathrm{cert}} < 178/10^{15}$, and the sign bridge between R and $\mathcal{B}$. The two Arb anchors are the only certificate-specific numerical assumptions in the final theorem. □

Supplementary Proposition 2 (Interval computation recorded by the archive)**.** *The archived Crunchy2 run for* `phase4_20260827_scalar_N150` *produced the Arb ball*

$$B_R = [1.9928311263803702 \times 10^{-14} \pm 1.5799937398052128 \times 10^{-13}], \tag{13}$$

whose outward endpoints are

$$-1.3807106271671758 \times 10^{-13} \le B_R \le 1.7792768524432498 \times 10^{-13}.$$

The upper endpoint is the value serialized as $\varepsilon_{0,\mathrm{cert}}$*. The same run produced the denominator lower endpoint* $D_{\mathrm{lower}} = 3.4465883518530394 \times 10^{-1}$*, serialized as* $D_{\mathrm{lower,cert}}$*.*

Supplementary Proposition 3 (Formal trust boundary)**.** *The exact residual and paper-operator Lean theorems depend on* `cert_arb_dmin_bound` *and* `cert_arb_sup_bound_le_eps0`*, together with the standard Mathlib principles* `propext`*,* `Classical.choice`*, and* `Quot.sound`*. The denominator theorem uses only the denominator anchor and the standard principles. The half-DC convention lemmas and paper-sign alignment theorem use no certificate-specific numerical axioms.*

Certified item	Julia or archive entry	Lean entry
Finite candidate and dyadic parameters	`pinn_a_coeffs_N150_phase4_20260827.txt`; `Certificate.jl`	`CertificatePhase4ScalarN150` `Phase420260827HalfDC.lean`
Path-A Hilbert transform on finite series	`src/FourierSeries.jl`	`Definitions.lean`; `HilbertOnTrigPoly.lean`
Path-B sampled Hilbert transform and aliasing	`src/FFTHilbert.jl`	`AliasingErrorBound.lean`; `FFTHilbertCorrectness.lean`
Taylor data, products, reciprocal, square root	`src/TaylorSeries.jl`; `src/GDerivatives.jl`	`LeibnizBound.lean`; `FaaDiBrunoBound.lean`; `TaylorSeriesCorrectness.lean`
Denominator positivity	`dmin_bound`; `staged_dlower.arbdump`	`DminBoundCorrectness.lean`; `cert_arb_dmin_bound`
Residual assembly and continuum scan	`src/ResidualBound.jl`; `staged_bound.arbdump`	`GridToSupNormTaylor.lean`; `ResidualAssemblyCorrectness.lean`; `cert_arb_sup_bound_le_eps0`
Paper theorem and audit	`phase4_scalar_n150_20260827_2m_results_20260829.tgz`	`MainPhase4ScalarN150` `Phase420260827HalfDC.lean`; `AuditPhase4ScalarN150HalfDC.lean`

1.4 Overview of the proof architecture

The proof has three conceptual layers. First, finite Fourier algebra constructs the candidate and handles the Hilbert transform whenever its input is a known trigonometric polynomial. Second, Julia and Arb compute outward-rounded interval enclosures for the nonlinear residual, add analytic corrections for unresolved Fourier modes, and use Taylor's theorem to pass from a grid to the whole torus. Third, Lean defines the same mathematical objects, proves the reusable analytic lemmas underlying the calculation, and checks the exact logical consequences of the exported numerical certificate.

The Hilbert-transform work is divided into two pathways.

Path A: finite trigonometric algebra. The input has a finite Fourier representation, so H acts mode-by-mode using Eq. (2). No sampling or aliasing correction is needed for this operation.

Path B: certified sampled transform. The input is represented by interval-enclosed values and derivatives on a uniform grid, rather than by a finite exact coefficient vector. A discrete Hilbert transform is evaluated by certified convolution, and a rigorous Fourier-tail estimate is added to enclose the continuous transform.

Path B is essential for $H[\partial_u G]$, because the square root and reciprocal in Eq. (3) generally give G an infinite Fourier series. The current Julia implementation also routes $H[yy_u]$ through Path B for uniformity, although yy_u is itself a finite trigonometric polynomial. For the present candidate its degree is at most 300, far below the Nyquist frequency of the two-million-point grid, so an exact coefficient treatment would have no unresolved modes. The generic Path-B routine does not exploit that special fact and adds its valid but conservative tail correction. This implementation choice does not project the product back to degree N.

1.5 The finite candidate and nonlinear mode generation

The final-stage PINN fields were sampled on a Fourier grid and projected onto finite trigonometric series. Their coefficients initialized Newton's method for a degree–160 Galerkin system. After polishing, the auxiliary fields were discarded and the scalar profile was truncated to degree $N = 150$, leaving 151 stored coefficients in Eq. (1). A high-precision non-interval diagnostic gave a residual of approximately 1.88×10^{-14}; this diagnostic is not used in the proof.

Only the candidate is truncated. The nonlinear residual is not projected onto the first N modes. In particular,

$$\cos(ku)\sin(\ell u) = \frac{1}{2}\big(\sin((k+\ell)u) + \sin((\ell-k)u)\big), \tag{14}$$

so products such as yy_u and $yH[y_u]$ can contain modes through degree $2N = 300$. The implementation forms the complete products. The non-polynomial functions D, D^{-1}, and G are not replaced by finite Fourier truncations at all; they are represented by interval Taylor data and global derivative bounds.

1.6 Path A: exact finite Fourier algebra

For a trigonometric polynomial

$$F(u) = \frac{\alpha_0}{2} + \sum_{k=1}^{K}\big(\alpha_k\cos(ku) + \gamma_k\sin(ku)\big), \tag{15}$$

Path A constructs

$$H[F](u) = \sum_{k=1}^{K}\big(\alpha_k\sin(ku) - \gamma_k\cos(ku)\big). \tag{16}$$

Differentiation and multiplication are likewise carried out by finite coefficient formulas. The inputs are Arb balls, so "exact" here means exact at the level of the finite Fourier representation: every arithmetic rounding error is enclosed by Arb, and there is no Fourier truncation or aliasing error attached to the operation.

Applying Eq. (16) to

$$y_u(u) = -\sum_{k=1}^{N} k a_k \sin(ku) \tag{17}$$

gives

$$H[y_u](u) = \sum_{k=1}^{N} k a_k \cos(ku). \tag{18}$$

This explicit function is denoted H_y in the implementation. It is used directly in D, G, and the local product yH_y.

For any derivative order $p \geq 1$, the same coefficient data give the rigorous global estimate

$$\|F^{(p)}\|_\infty \leq \sum_{k=1}^{K} k^p \big(|\alpha_k| + |\gamma_k|\big), \tag{19}$$

with the constant term included separately when $p = 0$. These finite sums are evaluated with outward rounding and seed the derivative-bound chain used by both pathways.

The corresponding Lean files are

`Definitions.lean`, `HilbertOnTrigPoly.lean`.

The first defines the Fourier-multiplier operator. The second proves the sine and cosine rules, compatibility with finite sums, and the bridge theorem

`periodicHilbert_yu_eq_Hyu`.

Thus the explicit finite expression in Eq. (18) is proved to equal the abstract Fourier-defined Hilbert transform of y_u.

1.7 Path B: certified discrete Hilbert transform

Let F be a smooth real 2π-periodic function and let $u_j = 2\pi j/N_{\text{grid}}$, where N_{grid} is even. The discrete Hilbert transform has Fourier multipliers $-i$ on positive discrete frequencies, $+i$ on negative frequencies, and zero at the constant and Nyquist modes. Julia realizes this operator as the circular convolution

$$H_{N_{\text{grid}}}[F](u_j) = \sum_{n=0}^{N_{\text{grid}}-1} h_n F(u_{j-n \bmod N_{\text{grid}}}), \tag{20}$$

with

$$h_n = \begin{cases} \dfrac{2}{N_{\text{grid}}} \cot\left(\dfrac{\pi n}{N_{\text{grid}}}\right), & n \text{ odd}, \\ 0, & n \text{ even}. \end{cases} \tag{21}$$

The cotangent formula is not a second definition of the continuous Hilbert transform. It is the inverse discrete Fourier transform of the discrete multiplier, and therefore an efficient real-space implementation of the same finite-dimensional operator.

The circular convolution is computed as a linear polynomial convolution followed by folding modulo $x^{N_{\text{grid}}} - 1$. Arb complex-ball polynomial multiplication encloses all arithmetic and transform rounding. The use of a fast convolution algorithm affects runtime, not the operator or the proof.

The sampled transform still differs from the continuous Hilbert transform because Fourier modes separated by a multiple of N_{grid} have the same samples. Write

$$\widehat{F}_k = \frac{1}{2\pi} \int_0^{2\pi} F(u) e^{-iku}\, du. \tag{22}$$

If F is periodic and $\|F^{(p)}\|_\infty \le C_p$ for an integer $p \ge 2$, integration by parts p times gives

$$|\widehat{F}_k| \le \frac{C_p}{|k|^p}, \qquad k \neq 0. \tag{23}$$

The discrete Fourier coefficient at a principal frequency is the sum of all continuous coefficients aliased into that class. The continuous and discrete Hilbert multipliers differ by at most two on an aliased mode. Combining this observation with Eq. (23), the two-sided Fourier tail, and the integral test yields

$$\left|H[F](u_j) - H_{N_{\mathrm{grid}}}[F](u_j)\right| \le \varepsilon_{\mathrm{alias}}(F) := \frac{4C_p}{(p-1)(N_{\mathrm{grid}}/2-1)^{p-1}}. \tag{24}$$

The symmetric interval $[-\varepsilon_{\mathrm{alias}}, \varepsilon_{\mathrm{alias}}]$ is added to every output ball. The resulting balls enclose the continuous Hilbert transform at the grid nodes, not merely its discrete approximation.

The transform is applied at every derivative level needed by the final Taylor argument. Since the periodic Hilbert transform commutes with differentiation under the smoothness hypotheses used here,

$$\partial_u^m H[F] = H[\partial_u^m F], \tag{25}$$

the same calculation supplies local enclosures of all required derivatives. Global derivative bounds for the output use

$$\|H[F]\|_\infty \le \frac{\pi^2}{3}\|F''\|_\infty, \tag{26}$$

which follows by two integrations by parts and $2\sum_{k\ge1} k^{-2} = \pi^2/3$.

The Lean file `AliasingErrorBound.lean` proves the DFT sampling identity, Fourier-decay and tail estimates, the bound Eq. (24), and the equivalence between the multiplier form and the cotangent convolution in Eq. (20). The file `FourierTailHilbert.lean` proves Eq. (26) and the supporting Fourier/Hilbert facts. Finally, `FFTHilbertCorrectness.lean` packages these results into correctness theorems for one transform and for the derivative-indexed Taylor data returned by Julia.

1.8 Interval Taylor data

An Arb ball $[m \pm r]$ represents the closed interval $[m - r, m + r]$. Elementary operations such as addition, multiplication, division away from zero, square root on a positive interval, sine, and cosine are rounded outward. Repeated use of the interval inclusion principle then guarantees that the final ball contains the exact result for the supplied dyadic inputs.

The rigor of the full verifier is not automatic from pointwise ball arithmetic alone. The code also supplies mathematical bounds for variation between grid points, unresolved Fourier modes, and nonlinear derivative growth. These analytic corrections are computed as Arb balls and inserted explicitly into the enclosure.

For a function F, Julia's `TaylorSeries` object stores

$$F^{(m)}(u_j) \in I_{j,m}(F), \quad 0 \le m \le M, \qquad C_m(F) \ge \|F^{(m)}\|_\infty, \quad 0 \le m \le M+1. \tag{27}$$

Products use the exact Leibniz identity

$$(FQ)^{(m)} = \sum_{r=0}^{m} \binom{m}{r} F^{(r)} Q^{(m-r)}, \tag{28}$$

both for local derivative balls and for global bounds. Reciprocals and square roots are propagated through Faà di Bruno formulas. A downward grid-and-Lipschitz pass can tighten an existing bound by

$$\|F^{(m)}\|_\infty \le \max_j \sup\{|x| : x \in I_{j,m}(F)\} + hC_{m+1}(F), \qquad h = \frac{2\pi}{N_{\text{grid}}}. \tag{29}$$

The new bound is retained only when its outward upper endpoint improves the previous rigorous estimate.

1.9 Worked evaluation of $yH[y_u]$

This subsection spells out one representative nonlinear term in the residual, namely

$$P(u) = y(u)H[y_u](u). \tag{30}$$

The notation is multiplicative: P is the pointwise product of y and the Hilbert transform of y_u. It is not the term $H[yy_u]$, which is computed separately. This distinction matters because $yH[y_u]$ is a local nonlinear product, while $H[yy_u]$ applies a nonlocal Fourier multiplier after multiplication.

For the certified candidate,

$$y(u) = \frac{a_0}{2} + \sum_{k=1}^{N} a_k \cos(ku), \qquad N = 150. \tag{31}$$

Path A differentiates and applies the periodic Hilbert transform mode-by-mode, giving

$$y_u(u) = -\sum_{k=1}^{N} k a_k \sin(ku), \qquad H[y_u](u) = \sum_{k=1}^{N} k a_k \cos(ku). \tag{32}$$

Thus P is itself a finite even trigonometric polynomial. Formally,

$$P(u) = \left(\frac{a_0}{2} + \sum_{k=1}^{N} a_k \cos(ku)\right)\left(\sum_{\ell=1}^{N} \ell a_\ell \cos(\ell u)\right), \tag{33}$$

and the identity

$$\cos(ku)\cos(\ell u) = \frac{1}{2}\big(\cos((k+\ell)u) + \cos((k-\ell)u)\big) \tag{34}$$

shows that modes through degree $2N$ may occur. No projection back to degree N is made.

The production verifier evaluates Eq. (30) through interval Taylor data. Let

$$u_j = \frac{2\pi(j-1)}{N_{\text{grid}}}, \qquad j = 1, \dots, N_{\text{grid}}, \qquad N_{\text{grid}} = 2{,}000{,}000.$$

For $F = y$ and $F = H[y_u]$, Julia first constructs a Taylor record consisting of Arb balls

$$F^{(m)}(u_j) \in I_{j,m}(F), \qquad 0 \le m \le M, \tag{35}$$

and global derivative bounds

$$C_m(F) \ge \|F^{(m)}\|_\infty, \qquad 0 \le m \le M+1. \tag{36}$$

For the certificate $M = 6$. The values in Eq. (35) are not ordinary floating-point samples: each entry is obtained by evaluating a finite trigonometric polynomial, or one of its coefficient derivatives, using

Arb enclosures of the coefficients and outward-rounded sine and cosine operations. The bounds in Eq. (36) come from the finite coefficient estimate in Eq. (19).

The Taylor record for $P = yH[y_u]$ is then built by the Leibniz rule. For each grid point and each $0 \le m \le M$,

$$P^{(m)}(u_j) \in I_{j,m}(P) := \sum_{r=0}^{m} \binom{m}{r} I_{j,r}(y)\, I_{j,m-r}(H[y_u]), \tag{37}$$

where the products and sums on the right are interval operations rounded outward. The corresponding initial global derivative bounds satisfy

$$\|P^{(m)}\|_\infty \le C_m^{\mathrm{seed}}(P) := \sum_{r=0}^{m} \binom{m}{r} C_r(y)\, C_{m-r}(H[y_u]), \qquad 0 \le m \le M+1. \tag{38}$$

The sum is evaluated outward, and its upper endpoint supplies the stored bound. After this product record is formed, the grid-and-Lipschitz pass computes

$$G_m(P) = \max_j \sup\{|x| : x \in I_{j,m}(P)\}, \qquad 0 \le m \le M, \tag{39}$$

and walks downwards from $m = M$ to $m = 0$, implementing the update

$$C_{M+1}(P) = C_{M+1}^{\mathrm{seed}}(P), \qquad C_m(P) = \min\{C_m^{\mathrm{seed}}(P),\, G_m(P) + hC_{m+1}(P)\}. \tag{40}$$

Here the minimum denotes comparison of outward upper endpoints, and $C_{m+1}(P)$ is the already updated next-order bound. The mean value theorem on each cell justifies the second candidate. This pass changes only the global bound vector; the local derivative balls remain unchanged.

For the production order $M = 6$, the pass can improve orders $6, 5, \ldots, 0$ but leaves $C_7(P)$ unchanged. This distinction determines its role in the final certificate: P enters the residual additively, and the final order-six scan uses its local derivative balls and its order-seven bound. Consequently, tightening the lower bounds of this particular product does not improve the final residual estimate in the current pipeline. The same tightening routine can affect the final estimate elsewhere, when tightened bounds feed subsequent products, nonlinear compositions, or truncation of a higher-order Taylor record.

Only after the product Taylor record has been assembled does the verifier use Taylor's theorem on a cell. For $u = u_j + t$, $0 \le t \le h$, with $h = 2\pi/N_{\mathrm{grid}}$, the product alone satisfies

$$|P(u)| \le \sum_{m=0}^{M} \frac{U_{j,m}(P) h^m}{m!} + \frac{C_{M+1}(P) h^{M+1}}{(M+1)!}, \tag{41}$$

where $U_{j,m}(P) = \sup\{|x| : x \in I_{j,m}(P)\}$. In the residual calculation, P is not scanned in isolation for the final theorem; its row-wise derivative balls are subtracted into the residual Taylor record, and the final scan is applied to the assembled residual. Equation (41) is nevertheless the local continuum principle that justifies the product record itself.

Julia pipeline for this term. The monolithic route in `ResidualBound.jl` constructs the term as follows:

```
y   = cosine_series(y_coeffs; prec=prec)
yu  = deriv(y)
Hyu = hilbert(yu)
```

```
ts_y_h = trigpoly_to_taylor(y, N_grid, M_h; prec=prec)
ts_Hyu = trigpoly_to_taylor(Hyu, N_grid, M_order; prec=prec)
ts_y   = truncate_order(ts_y_h, M_order)

ts_y_Hyu = ts_y * ts_Hyu
ts_R = Arb(mu; prec=prec) * ts_Hyu - ts_y - ts_y_Hyu - ts_Hyyu
```

The first three lines are finite Fourier algebra. The two `trigpoly_to_taylor` calls evaluate derivative towers at the global grid points and attach derivative sup-norm bounds. The multiplication line is the Leibniz construction in Eq. (37); it does not multiply two already-scanned cell supremum bounds. Internally, multiplication ends with `return tighten_sup_bounds!(out)` in `TaylorSeries.jl`.

The sharded Crunchy2 route performs the same operations on contiguous blocks of rows. The following excerpt uses the setup record `su`, precision `p`, and accumulator defined by that driver:

```
for si in shard_range
    jr = su.shards[si]
    ts_y_h = trigpoly_to_taylor(su.y, su.N_grid, su.M_h;
                                prec=p, j_range=jr)
    ts_Hyu = trigpoly_to_taylor(su.Hyu, su.N_grid, M_order;
                                prec=p, j_range=jr)
    ts_y   = truncate_order(ts_y_h, M_order)

    ts_y_Hyu = ts_y * ts_Hyu
    _acc!(acc_yHyu, ts_y_Hyu.derivs, M_order)
    write_matrix_slices(store, "yHyu", si, ts_y_Hyu.derivs)
end
```

The accumulator records per-derivative maxima needed for the later grid-and-Lipschitz tightening. In the scalar replay stage, the same Leibniz formula is applied to the global sup-bound vectors, then tightened using maxima merged over the entire grid:

```
sb_y_Hyu = _leibniz_sup_bounds(sb_y, sb_Hyu,
                               M_order, p, su.binom_M)
_tighten_from_max!(sb_y_Hyu, st.acc_yHyu.ubs,
                   M_order, su.h_arb, p)
```

These are the global bounds used in the replay; maxima from a single shard alone do not certify a bound on the entire period. The loop inside `_tighten_from_max!` visits `M:-1:0`, leaving `sb_y_Hyu[M+2]` unchanged. In the final scan, the stored `yHyu` rows are read back and subtracted in the same algebraic order as the monolithic residual assembly.

Lean interface for this term. Lean defines the functions appearing above as finite sums:

$$y(a,u) = \sum_k a_k \cos(ku), \tag{42}$$

$$y_u(a,u) = -\sum_k k a_k \sin(ku), \tag{43}$$

$$H_y(a,u) = \sum_k k a_k \cos(ku). \tag{44}$$

The theorem `periodicHilbert_yu_eq_Hyu` identifies the explicit finite sum H_y with the abstract Fourier-defined periodic Hilbert transform of y_u. Lean's sum uses the full constant coefficient; the generated half-DC certificate therefore stores $a_0/2$ in its zeroth coefficient to match Julia's convention.

The generic proof in `CertificateAxioms.lean` asks its assumed Arb interface for Taylor records of y and H_y, and applies the Taylor-series multiplication certificate. Schematically (with angle brackets denoting Lean tuple destructuring and omitted arguments denoted by ellipses):

```
obtain <ts_y, h_y, hh_y> :=
  cert_arb_trigpoly_to_taylor (y a_cert) M N_fft hN_fft 0 ...

obtain <ts_Hyu, h_Hyu, hh_Hyu> :=
  cert_arb_trigpoly_to_taylor (Hyu a_cert) M N_fft hN_fft 0 ...

obtain <ts_yHyu, h_yHyu, hh_yHyu> :=
  cert_arb_ts_mul h_y h_Hyu (hh_y.trans hh_Hyu.symm)
```

The resulting semantic statement is that `ts_yHyu` is a valid Taylor record for

$$u \mapsto y(a_{\mathrm{cert}}, u)\, H_y(a_{\mathrm{cert}}, u).$$

During residual assembly this generic proof subtracts the record after forming $\mu H_y - y$. Thus the formal proof tracks the same operator order as the Julia computation: y, H_y, product yH_y, then subtraction into the residual.

The dedicated half-DC theorem for the two-million-point run uses a different numerical interface, in the module `CertificatePhase4ScalarN150Phase420260827HalfDCAxioms`: it assumes the two numerical certificates `cert_arb_dmin_bound` and `cert_arb_sup_bound_le_eps0`, then proves periodic extension of the residual bound in Lean. It does not independently replay the Arb product or its tightening pass. The generic excerpt above describes the term's Taylor-record interface, not the numerical dependency chain of that dedicated theorem.

Performance role. The archived Crunchy2 logs do not isolate a separate timer for `yHyu`. Its cost is included in the first sharded pointwise stage, where each worker also computes yy_u, denominator data, reciprocal data, and the capillary auxiliary records. For each grid row, the `yHyu` product uses the $M + 1$ stored derivative columns and the triangular Leibniz sums, hence $O(M^2)$ Arb operations per grid point with $M = 6$. At order six this is 28 Leibniz summands per row, or 56 million across the full grid. The grid reduction for tightening visits seven derivative columns, or 14 million entries; its scalar downward pass has seven steps. These operation counts do not determine separate wall times, which were not recorded for this term.

In the production run the eight first-stage workers ran concurrently. Their wall times ranged from 1:56:25 to 2:14:58, with peak resident memory between 4.972 GB and 9.324 GB. The subsequent serial stages took 1:22.08 for the merge, 21:33.92 for the certified FFT/Hilbert stage, and 11:49.78 for the final residual scan. Since the first-stage workers overlap, the effective post-warmup wall time is approximately the slowest worker plus these serial stages, about 2:49:44. Sharding keeps `yHyu` as 1024-row Taylor blocks in memory during stage 1 and as exact Arb slices on disk afterward; the full two-million-row product matrix is never required to reside in RAM at once.

1.10 Positivity of the capillary denominator

The reciprocal terms are not evaluated until D has been proved positive. Set

$$A = y_u, \qquad B = 1 + H[y_u], \qquad D = \sqrt{A^2 + B^2}. \tag{45}$$

At each grid point, Arb computes a lower endpoint $\underline{D}_j$ for $D(u_j)$. The reverse triangle inequality gives

$$|D(u) - D(v)| \le |A(u) - A(v)| + |B(u) - B(v)|. \tag{46}$$

Consequently, if $L_A \ge \|A'\|_\infty$ and $L_B \ge \|B'\|_\infty$, then

$$d_{\mathrm{Lip}} := \min_j \big(\underline{D}_j - h(L_A + L_B)\big) \le \inf_u D(u). \tag{47}$$

This proof neither differentiates nor divides by D, so it does not presuppose the positivity it establishes. The computation stops unless outward-rounded arithmetic proves $d_{\mathrm{Lip}} > 0$.

After positivity is known, Julia constructs Taylor data for D and obtains a second lower bound from the local derivatives and the Taylor remainder. The implementation selects the ball having the larger rigorous lower endpoint. Both candidates are valid lower bounds; selecting the larger one only sharpens the subsequent reciprocal estimates.

1.11 Higher derivatives of D and G

Direct Faà di Bruno estimates for many derivatives of a square root and reciprocal can become unnecessarily large. The verifier therefore also uses algebraic recurrences. Let $S = A^2 + B^2 = D^2$. Differentiating $D^2 = S$ m times and isolating the two endpoint terms gives

$$2DD^{(m)} = S^{(m)} - \sum_{r=1}^{m-1} \binom{m}{r} D^{(r)} D^{(m-r)}. \tag{48}$$

If $D \ge d > 0$, then

$$\|D^{(m)}\|_\infty \le \frac{1}{2d} \left(\|S^{(m)}\|_\infty + \sum_{r=1}^{m-1} \binom{m}{r} \|D^{(r)}\|_\infty \|D^{(m-r)}\|_\infty \right). \tag{49}$$

Similarly, the identity $GD = B$ gives

$$G^{(m)} D = B^{(m)} - \sum_{r=0}^{m-1} \binom{m}{r} G^{(r)} D^{(m-r)}, \tag{50}$$

and therefore

$$\|G^{(m)}\|_\infty \le \frac{1}{d} \left(\|B^{(m)}\|_\infty + \sum_{r=0}^{m-1} \binom{m}{r} \|G^{(r)}\|_\infty \|D^{(m-r)}\|_\infty \right). \tag{51}$$

The quantities S and B are finite trigonometric polynomials, so their derivative bounds come directly from Eq. (19). The recurrence avoids replacing D or G by a finite Fourier series. Julia compares these algebraic bounds with the valid composition-based bounds and uses the tighter outward endpoint where needed for the top derivatives in Path B.

1.12 Residual assembly and passage from the grid to the torus

The local interval records are composed in the same order as Eq. (4): finite data for y, y_u, and $H[y_u]$; Leibniz products; the Path-B transform of yy_u; construction of D, D^{-1}, and G; differentiation of y_u/D and G; the Path-B transform of $\partial_u G$; and the final linear combination. For every grid node and every $0 \le m \le M$, the result is an interval

$$R^{(m)}(u_j) \in I_{j,m}(R), \tag{52}$$

together with $C_{M+1}(R) \ge \|R^{(M+1)}\|_\infty$.

For $u = u_j + t$, $0 \le t \le h$, Taylor's theorem yields

$$|R(u)| \le \sum_{m=0}^{M} \frac{U_{j,m}(R) h^m}{m!} + \frac{C_{M+1}(R) h^{M+1}}{(M+1)!}, \tag{53}$$

where $U_{j,m}(R) = \sup\{|x| : x \in I_{j,m}(R)\}$. The outward-rounded maximum of the right-hand side over all cells is the certified ε_*. This is the step that converts finitely many computations into a statement about every point of the continuous periodic domain. With $M = 6$, the final remainder scales as h^7.

1.13 Julia/Arb implementation

The package `BabenkoRigorous` implements the numerical proof. Its principal files have the following roles.

`FourierSeries.jl`. Defines the finite `TrigPoly` representation and the Path-A operations: evaluation, differentiation, Hilbert transform, multiplication, and coefficient-based supremum bounds.

`TaylorSeries.jl`. Defines local derivative matrices and global derivative-bound vectors; implements Taylor supremum and infimum scans, Leibniz products, reciprocal and square-root composition, downward bound tightening, denominator positivity, and the algebraic D/G recurrences.

`GDerivatives.jl`. Constructs the Taylor representation of $G = (1 + H[y_u])/D$ from the certified denominator and reciprocal.

`FFTHilbert.jl`. Implements Path B by Arb complex-polynomial convolution with the cotangent kernel, adds Eq. (24) to each output ball, and lifts the transform through all derivative levels using Eq. (25) and the global bound Eq. (26).

`ResidualBound.jl`. Assembles the complete gravity–capillary residual and applies the final Taylor scan. In the implemented route, $H[yy_u]$ and $H[\partial_u G]$ both pass through `fft_hilbert_series`; only the latter intrinsically requires the non-polynomial pathway.

`ShardStore.jl` **and** `Sharded.jl`. Reorganize the same computation into exact on-disk shards so that a large grid does not require all Taylor matrices to be resident simultaneously.

`Certificate.jl`. Converts binary inputs and outward endpoints to exact rational literals and emits the generated Lean data module. The current formatter emits no axioms; the two numerical anchors live in a separate certificate-specific Lean module.

Verifier order. The production verifier follows the order below. This order is useful for audit because every later division or Hilbert transform consumes bounds created earlier in the list.

1. Read the 151 stored coefficients and construct `TrigPoly` objects for y, y_u, and $H[y_u]$.
2. Convert the finite series into Taylor records on the N_{grid}-point grid, retaining enough derivative orders to support the final $M = 6$ Taylor scan.
3. Form the polynomial products $yH[y_u]$ and yy_u by the Leibniz rule, and route yy_u through the certified sampled Hilbert transform.
4. Prove a positive lower bound for $D = \sqrt{y_u^2 + (1 + H[y_u])^2}$ before any reciprocal is evaluated.
5. Build D^{-1}, $G = (1 + H[y_u])/D$, $\partial_u(y_u/D)$, and $\partial_u G$ as Taylor records with outward-rounded derivative bounds.
6. Apply the certified sampled Hilbert transform to $\partial_u G$, using the supplied high-derivative bounds to add the aliasing radius and the Fourier-tail derivative bounds.
7. Assemble the residual record and scan all cells with Eq. (53); write the residual ball, the denominator lower ball, and the generated certificate.

Most local arithmetic operations are ordinary point operations lifted to Arb balls, but the proof also depends on the explicitly programmed analytic corrections in Eqs. (24), (29), (47), and (53). Arb automatically encloses numerical rounding; it does not invent these mathematical bounds.

1.14 Memory-sharded computation

At $N_{\text{grid}} = 2{,}000{,}000$, storing every intermediate matrix at once is unnecessarily expensive. The sharded implementation exploits the structure of the calculation:

1. Compute coefficient-level derivative bounds once from the finite trigonometric data.
2. Pointwise nonlinear recurrences and min/max reductions are streamed over contiguous grid shards.
3. Each global Hilbert transform is processed one derivative column at a time. Exact Arb column slices are stored on disk.
4. The final residual is reassembled shard by shard and the Taylor-cell maximum is reduced in ascending grid order.

Intermediate balls are serialized with Arb's exact dump/load representation, including mantissa, exponent, and radius; decimal text is not used for this round trip. The reduction order matches the monolithic loops, and repository tests compare the sharded and monolithic paths on smaller grids. Sharding therefore changes memory use and scheduling, not the mathematical enclosure.

The successful two-million-point calculation used shards of 1024 points. The first stage ran on eight workers with four Julia threads per worker; the main transform and final stages used eight threads. The stages consisted of parallel pointwise reductions, merging of the worker states, derivative-by-derivative certified convolutions, and a final sharded residual scan. Separate operating-system processes between major stages released the memory associated with completed stages and left exact state dumps for reproducibility.

1.15 Lean definitions and reusable analytic library

The Lean project `WaveCAP` defines the mathematical operator rather than treating it as an uninterpreted symbol. For a real function f, `Definitions.lean` sets

$$H[f](u) = \operatorname{Re} \sum_{n \in \mathbb{Z}} \bigl(-i \operatorname{sgn}(n)\bigr) \widehat{f}_n e^{inu}, \tag{54}$$

with multiplier zero at $n = 0$. The same file defines the finite profile, its derivative, the explicit H_y, D, G, and R from Eq. (4). The coefficient type $\mathrm{Fin}(N) \to \mathbb{R}$ makes every profile sum finite.

The reusable formal library mirrors the proof architecture:

- **Hilbert transform.** `HilbertOnTrigPoly.lean` proves Path A and $H[y_u] = H_y$. `FourierTailHilbert.lean` proves the $\pi^2/3$ estimate and mode facts. `HilbertCommutesDeriv.lean` proves derivative commutation.
- **Aliasing and Path B.** `AliasingErrorBound.lean` proves the sampled DFT aliasing theorem and the cot-kernel equivalence. The file `FFTHilbertCorrectness.lean` states the correctness contract for the interval transform and its Taylor-series lift.
- **Derivative calculus.** `LeibnizBound.lean` and `FaaDiBrunoBound.lean` prove the product and composition bounds used by Julia.
- **Taylor data.** `TaylorRemainder.lean` and `GridToSupNormTaylor.lean` prove the local remainder and the grid-to-continuum theorem. The file `TaylorSeriesCorrectness.lean` defines the semantic invariant `IsTaylorSeriesOf` and proves that the arithmetic operations preserve it. `SeriesCorrectness.lean` covers the square-root and reciprocal recurrences, while `TightenSupBoundsLemma.lean` justifies the downward tightening pass.
- **Denominator and capillary chain.** `DminBoundCorrectness.lean` formalizes the positivity calculation, and `TightDGSupBounds.lean` proves Eqs. (49)–(51).
- **Residual assembly.** `ResidualAssemblyCorrectness.lean` composes the Path-A and Path-B records into the full residual, proves periodicity, and proves the general extension of a one-period bound to every real argument.

These are generic theorems about the mathematics and data-structure contracts of the pipeline. They are useful independently of the particular numerical profile.

1.16 Exact certificate data and convention alignment

The generated module

`CertificatePhase4ScalarN150Phase420260827HalfDC.lean`

contains 151 exact rational coefficients and exact rational representations of μ, β, N_{grid}, M, the residual upper endpoint, and the denominator lower endpoint. In particular,

$$\mu_{\mathrm{cert}} = \frac{1538048895284245}{1125899906842624}, \tag{55}$$

$$\beta_{\mathrm{cert}} = \frac{2233821443972785}{9007199254740992}, \tag{56}$$

and

$$\varepsilon_{0,\mathrm{cert}} = \frac{66570650447636094343693612891855023841}{37414441915671114706014331717536845303191873100185 6}, \tag{57}$$

$$D_{\mathrm{lower,cert}} = \frac{30024018999569627600479976468787845960585}{87112285931760246646623899502532662132736}. \tag{58}$$

Thus moving the certificate into Lean introduces no decimal-rounding step.

Julia and Lean use different storage conventions for the constant cosine mode. Julia evaluates the first source entry as $a_0/2$, while the generic Lean function `y` uses its zeroth stored entry directly. The certificate therefore stores

$$\widehat{a}_0 = a_0/2, \qquad \widehat{a}_k = a_k \quad (k \geq 1). \tag{59}$$

The generic module `HalfDCConvention.lean` proves the two coefficient cases and proves that this adapter preserves y_u, H_y, D, and G. The certificate-specific theorem checks the exact zeroth-entry identity. The source vector is not also encoded as a second complete Lean vector, so the present formal theorem does not compare two named 151-component vectors; the generated certificate already contains the converted vector.

The audit module `AuditPhase4ScalarN150HalfDC.lean` separately proves the sign identity in Eq. (10) and the corresponding equality of absolute values. Both the half-DC identities and the residual sign bridge are independent of the numerical trust anchors.

1.17 The numerical trust boundary in Lean

The present certificate does not serialize the millions of local derivative balls, all global derivative vectors, or the intermediate convolution enclosures needed for Lean to reconstruct the two-million-point run. The certificate-specific numerical interface therefore consists of exactly two assumptions recording the final Arb conclusions:

$$\mathsf{A}_D: \quad D_{\mathrm{lower,cert}} > 0 \quad \text{and} \quad \forall u \in \mathbb{R}, \quad D_{\mathrm{lower,cert}} \leq D(a_{\mathrm{cert}}; u), \tag{60}$$

$$\mathsf{A}_R: \quad \mathsf{A}_D \Longrightarrow \forall u \in [0, 2\pi], \quad |R(a_{\mathrm{cert}}, \mu_{\mathrm{cert}}, \beta_{\mathrm{cert}}; u)| \leq \varepsilon_{0,\mathrm{cert}}. \tag{61}$$

Their Lean names are

`cert_arb_dmin_bound`, `cert_arb_sup_bound_le_eps0`.

They are declared in

`CertificatePhase4ScalarN150Phase420260827HalfDCAxioms.lean`.

This distinction is important. Lean proves the universal analytic ingredients used by the verifier, but the current certificate-facing theorem does not instantiate those generic results from serialized grid matrices. Instead, it imports Eqs. (60)– (61) as the explicit numerical trust boundary. Accordingly, the result should be described as a machine-checked theorem with two externally computed Arb anchors, not as a replay of every interval operation inside Lean.

From the two anchors Lean performs the remaining deductions:

1. It projects the global denominator lower bound from A_D.
2. It applies A_R on $[0, 2\pi]$.
3. It proves periodicity of the residual and extends the bound to all $u \in \mathbb{R}$.

4. It verifies by exact rational normalization that $\varepsilon_{0,\mathrm{cert}} < 178/10^{15}$ and $3446588/10^7 < D_{\mathrm{lower,cert}}$.

5. It transfers the absolute residual theorem to the manuscript operator using Eq. (10).

The corresponding public modules are

`MainPhase4ScalarN150Phase420260827HalfDC.lean`,
`AuditPhase4ScalarN150HalfDC.lean`.

The axiom audit for the exact and decimal residual theorems reports the standard Mathlib principles `propext`, `Classical.choice`, and `Quot.sound`, together with precisely the two numerical anchors above. The denominator theorem uses only the first numerical anchor. The half-DC and sign-alignment results use no certificate-specific numerical axioms.

1.18 Numerical result and interpretation of precision

The successful Crunchy2 computation used

$$N = 150, \qquad N_{\mathrm{grid}} = 2{,}000{,}000, \tag{62}$$

$$M = 6, \qquad p_{\mathrm{Arb}} = 128 \text{ bits}, \tag{63}$$

with shard size 1024. The final Arb ball was reported as

$$\mathrm{mid}(B_R) = 1.9928311263803702 \times 10^{-14}, \tag{64}$$

$$\mathrm{rad}(B_R) = 1.5799937398052128 \times 10^{-13}, \tag{65}$$

$$\inf B_R = -1.3807106271671758 \times 10^{-13}, \tag{66}$$

$$\sup B_R = 1.7792768524432498 \times 10^{-13}. \tag{67}$$

Here B_R is the ball returned by the supremum-bound computation; it is not an enclosure of the range of R, which explains why its lower endpoint may be negative. The theorem uses the outward upper endpoint $1.7792768524432498 \times 10^{-13}$, not the midpoint or radius separately. The denominator calculation returned the outward lower endpoint

$$D_{\mathrm{lower}} = 3.4465883518530394 \times 10^{-1}. \tag{68}$$

The run log ends with `ALL_STAGES_COMPLETE`, and the final stage has zero exit status.

The phrase "near machine precision" refers here to the practical accuracy floor of the double-precision candidate and the complete nonlinear residual, not to the rounding unit of the 128-bit Arb calculation. IEEE double precision has machine epsilon $2^{-52} \approx 2.22 \times 10^{-16}$ and unit roundoff $2^{-53} \approx 1.11 \times 10^{-16}$. Cancellation, derivative amplification, conditioning, and the storage of the candidate coefficients can raise the attainable residual above these raw constants. The 128-bit Arb precision, roughly 38 decimal digits, makes arithmetic rounding in the verification negligible relative to the analytic Taylor and aliasing corrections; it does not force the final certificate radius to be 10^{-38}.

1.19 Reproducibility and file map

The paths and production commands in this section preserve the original research checkout and Crunchy2 environment. In the paper's publication repository, `BabenkoRigorous/` corresponds to `Julia/`, `WaveCAP/WaveCAP/` corresponds to `Lean/WaveCAP/`, and `wave-cap/WaveCAP/` corresponds to `Lean/`. The repository README gives build and verification commands for the publication layout.

The archived run directory

`BabenkoRigorous/results/phase4_scalar_n150_20260827_2m_20260829`

contains the run description and checksums for the raw logs, stage logs, coefficient file, Arb endpoint dumps, scripts, and generated Lean certificate. The installed half-DC certificate was compared byte-for-byte with the generated artifact. The dedicated Lean theorem and audit modules were checked directly with the pinned Lean and Mathlib version `v4.29.0-rc6`.

The principal archived artifacts have the following SHA-256 checksums:

`4d65357a64a0ccfbf0ed67f8dd0e1970f0ceb549a8cbf647d8490e4ac48abb07`
`ee8fe5db1ea3bbd6e37d0888fdadc00c9c9e1d6ce81cb9d46ae3e4db8ffcdf43`

for the run archive and the half-DC alignment archive, respectively. The installed generated half-DC Lean certificate has SHA-256 checksum

`2190a52f3520ab0b15683eab592ee1105797d737a22f2e24a869130f5a431ed2`

Crunchy2 execution table. The following table is transcribed from the archived `/usr/bin/time -v` logs and from the stage summaries printed by `stage_run_phase4_scalar_n150_20260827.jl`. The eight `p1part` workers were launched concurrently, so their wall times should not be summed. The total production wall time after warmup is approximately the slowest `p1part` worker plus the three serial stages.

Table 1: Crunchy2 stage timing and memory for the certified two-million-point run.

Log file	Threads	Wall time	Julia elapsed	Peak RSS
`p1part_1of8.log`	4	1:57:25	6959.9 s	5.019 GB
`p1part_2of8.log`	4	2:14:58	8007.8 s	9.242 GB
`p1part_3of8.log`	4	1:56:25	6895.6 s	4.972 GB
`p1part_4of8.log`	4	2:07:07	7541.7 s	9.287 GB
`p1part_5of8.log`	4	2:13:04	7897.6 s	9.324 GB
`p1part_6of8.log`	4	1:58:34	7032.1 s	5.088 GB
`p1part_7of8.log`	4	2:09:00	7655.0 s	9.267 GB
`p1part_8of8.log`	4	2:13:52	7944.0 s	9.283 GB
`p1merge.log`	8	1:22.08	0.7 s	0.481 GB
`fft.log`	8	21:33.92	1212.4 s	5.149 GB
`s3.log`	8	11:49.78	628.2 s	0.477 GB

The final s3 log reports

```
bound midpoint = +1.9928311263803702e-14
bound radius   = 1.5799937398052128e-13
bound interval = [-1.3807106271671758e-13, +1.7792768524432498e-13]
bound ubound   = 1.7792768524432498e-13
D_lower lbound = 3.4465883518530394e-01
```

and the top-level run log ends with ALL_STAGES_COMPLETE.

Production command. On crunchy2, from the BabenkoRigorous directory, the archived run is reproduced by

```
WORK=/scratch/ms14479/phase4_scalar_n150_20260827_2m_retry_work
LOGDIR=/scratch/ms14479/phase4_scalar_n150_20260827_2m_retry_logs
RUNTIME=apptainer
SIF=/home/tb97/transit-cap-ccf/cap-ccf.sif
BINDPATH=/scratch:/scratch
WORKER_THREADS=4
MAIN_THREADS=8
scripts/p1_parallel_run_phase4_scalar_n150_20260827.sh \
  "$WORK" 1024 2000000 8 "$LOGDIR"
```

The first run writes the non-half-DC generated certificate and the two Arb dumps staged_bound.arbdump and staged_dlower.arbdump. The half-DC Lean certificate used by the paper is then re-emitted from the existing dumps, without rerunning the two-million-point interval calculation, by

```
apptainer exec --bind /scratch:/scratch \
  /home/tb97/transit-cap-ccf/cap-ccf.sif \
  julia --startup-file=no --compiled-modules=no --project=. \
  scripts/reemit_phase4_n150_half_dc_certificate.jl "$WORK"
```

Lean verification commands. After the generated half-DC certificate is installed in WaveCAP/WaveCAP/, the certificate-facing targets are checked by

```
cd wave-cap/WaveCAP
lake build
lake env lean WaveCAP/MainPhase4ScalarN150Phase420260827HalfDC.lean
lake env lean WaveCAP/AuditPhase4ScalarN150HalfDC.lean
```

The direct lake env lean calls are included because the dedicated half-DC headline and audit modules are not exercised by the default library root in the current repository layout.

For readers following the implementation, the Julia source names below are relative to BabenkoRigorous/. The Lean source names are relative to WaveCAP/WaveCAP/.

- **Path A.** Julia: src/FourierSeries.jl. Lean: Definitions.lean and HilbertOnTrigPoly.lean.
- **Path B and the cot kernel.** Julia: src/FFTHilbert.jl. Lean: AliasingErrorBound.lean, FourierTailHilbert.lean, and FFTHilbertCorrectness.lean.

- **Nonlinear Taylor data and denominator.** Julia: `src/TaylorSeries.jl` and `src/GDerivatives.jl`. Lean: `TaylorSeriesCorrectness.lean`, `SeriesCorrectness.lean`, `DminBoundCorrectness.lean`, and `TightDGSupBounds.lean`.
- **Residual and continuum bound.** Julia: `src/ResidualBound.jl`. Lean: `TaylorRemainder.lean`, `GridToSupNormTaylor.lean`, and `ResidualAssemblyCorrectness.lean`.
- **Large-grid organization.** Julia: `src/ShardStore.jl`, `src/Sharded.jl`, and `scripts/stage_run_phase4_scalar_n150_20260827.jl`.
- **Certificate and theorem.** Julia: `src/Certificate.jl`. Lean: the generated half-DC certificate, its certificate-specific axiom module, the corresponding `Main` module, and `AuditPhase4ScalarN150HalfDC.lean`.

These components yield a reproducible continuum residual bound for the profile in Eq. (1). They state the mathematical pathways, numerical enclosures, formal definitions, and remaining numerical trust boundary separately.